Build Automation

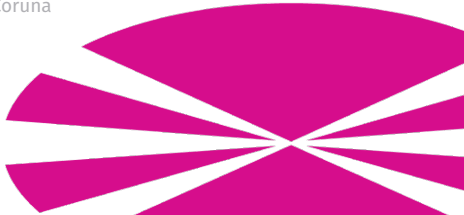
Ferramentas de Desenvolvemento

Javier Parapar López

Grao en Enxeñaría Informática · Facultade de Informática · Universidade da Coruña

Curso 2026/2027

GEI GRADO EN ENXEÑARÍA INFORMÁTICA



Where we are going

1. Why a build tool
2. Maven
3. Dependencies
4. Gradle
5. Working this way

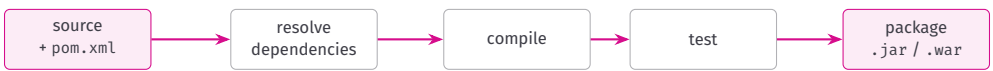
SECTION 1

Why a build tool

1

The build is a program too

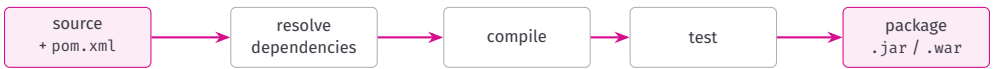
Between the source you write and the thing a user runs there is a sequence of steps. Build automation is writing that sequence down once, so that a **machine** performs it instead of a person remembering it.



The steps are always the same ones, which is exactly why they can be automated: resolve what the code needs, compile it, run the tests, package the result, and — later — publish and deploy it.

The build is a program too

Between the source you write and the thing a user runs there is a sequence of steps. Build automation is writing that sequence down once, so that a **machine** performs it instead of a person remembering it.

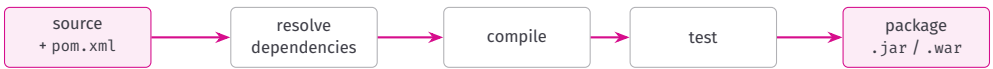


The steps are always the same ones, which is exactly why they can be automated: resolve what the code needs, compile it, run the tests, package the result, and — later — publish and deploy it.

The point is not saving keystrokes. It is that a written build is **repeatable**, **inspectable** and **reviewable** like any other code in the repository — and that it runs the same way when you are not the one running it.

The build is a program too

Between the source you write and the thing a user runs there is a sequence of steps. Build automation is writing that sequence down once, so that a **machine** performs it instead of a person remembering it.



The steps are always the same ones, which is exactly why they can be automated: resolve what the code needs, compile it, run the tests, package the result, and — later — publish and deploy it.

The point is not saving keystrokes. It is that a written build is **repeatable**, **inspectable** and **reviewable** like any other code in the repository — and that it runs the same way when you are not the one running it.

"It works on my machine" is not an excuse. It is a bug report about the build.

One command, on anybody's machine

The real test of a build is not that it works, but that **somebody else can run it**: a teammate who joined yesterday, a marker, or a server that has never seen your laptop.

What that requires

Every dependency **declared**, none installed by hand · every version **written down**, none floating · the tool itself **pinned** · **no step outside the script**, not even a click in the IDE

One command, on anybody's machine

The real test of a build is not that it works, but that **somebody else can run it**: a teammate who joined yesterday, a marker, or a server that has never seen your laptop.

What that requires

Every dependency **declared**, none installed by hand · every version **written down**, none floating · the tool itself **pinned** · **no step outside the script**, not even a click in the IDE

What it buys you

A **new machine** builds it in one command · **CI** runs exactly what you run, so a red build means something · and a **version from six months ago** still builds, which is what *maintenance* means

One command, on anybody's machine

The real test of a build is not that it works, but that **somebody else can run it**: a teammate who joined yesterday, a marker, or a server that has never seen your laptop.

What that requires

Every dependency **declared**, none installed by hand · every version **written down**, none floating · the tool itself **pinned** · **no step outside the script**, not even a click in the IDE

What it buys you

A **new machine** builds it in one command · **CI** runs exactly what you run, so a red build means something · and a **version from six months ago** still builds, which is what *maintenance* means

This is the contract with *Lesson 5*: Jenkins will not do anything clever. It will check out your branch and run `mvn verify`. Everything the pipeline needs to know has to be **inside the build**, because the server has nothing else.

One command, on anybody's machine

The real test of a build is not that it works, but that **somebody else can run it**: a teammate who joined yesterday, a marker, or a server that has never seen your laptop.

What that requires

Every dependency **declared**, none installed by hand · every version **written down**, none floating · the tool itself **pinned** · **no step outside the script**, not even a click in the IDE

What it buys you

A **new machine** builds it in one command · **CI** runs exactly what you run, so a red build means something · and a **version from six months ago** still builds, which is what *maintenance* means

This is the contract with *Lesson 5*: Jenkins will not do anything clever. It will check out your branch and run `mvn verify`. Everything the pipeline needs to know has to be **inside the build**, because the server has nothing else.

So the useful question about any build step: would it survive on a machine with nothing but a JDK, a clone of the repository, and no human?

Everybody has one, and they all do the same four things

Make, Ant, **Maven**, **Gradle**, sbt, Bazel, **npm**, Cargo, Go modules, pip/Poetry, MSBuild, CMake… The syntax is different every time; the model is not.

THE IDEA	MAVEN	NPM
the build file	`pom.xml`	`package.json`
the unit you depend on	artifact (`group:artifact:version`)	package (`name@version`)
where it comes from	Maven Central	the npm registry
the local cache	`~/.m2/repository`	`node_modules` (per project)
what you run	`mvn verify`	`npm run build`
what is pinned	the versions in the POM	`package-lock.json`

Everybody has one, and they all do the same four things

Make, Ant, **Maven**, **Gradle**, sbt, Bazel, **npm**, Cargo, Go modules, pip/Poetry, MSBuild, CMake… The syntax is different every time; the model is not.

THE IDEA	MAVEN	NPM
the build file	`pom.xml`	`package.json`
the unit you depend on	artifact (`group:artifact:version`)	package (`name@version`)
where it comes from	Maven Central	the npm registry
the local cache	`~/.m2/repository`	`node_modules` (per project)
what you run	`mvn verify`	`npm run build`
what is pinned	the versions in the POM	`package-lock.json`

Your assignment uses **both**: Maven for the Spring Boot back end, and for the React front end **pnpm** — npm's model, with a shared package store — which Maven itself runs. That is the ordinary situation, and it is why this lesson looks at the two side by side.

Everybody has one, and they all do the same four things

Make, Ant, **Maven**, **Gradle**, sbt, Bazel, **npm**, Cargo, Go modules, pip/Poetry, MSBuild, CMake… The syntax is different every time; the model is not.

THE IDEA	MAVEN	NPM
the build file	`pom.xml`	`package.json`
the unit you depend on	artifact (`group:artifact:version`)	package (`name@version`)
where it comes from	Maven Central	the npm registry
the local cache	`~/.m2/repository`	`node_modules` (per project)
what you run	`mvn verify`	`npm run build`
what is pinned	the versions in the POM	`package-lock.json`

Your assignment uses **both**: Maven for the Spring Boot back end, and for the React front end **pnpm** — npm's model, with a shared package store — which Maven itself runs. That is the ordinary situation, and it is why this lesson looks at the two side by side.

Learn the model once and the next tool takes an afternoon. Learn only the commands and every new project is a fresh mystery.

SECTION 2

Maven

2

Maven

- A **build and project management** tool from the Apache Foundation, and the default in the Java world for twenty years
- Its bet is convention over configuration: Maven already knows where your sources live, where the tests are, and what a build does — you only describe what is *different*
- So a `pom.xml` is not a script. It is a **description of the project**, and Maven derives the build from it
- Which is why any Maven project, anywhere, is navigable by anyone who has seen one before



current stable **3.9.16** (May 2026)
Maven 4 still in release candidates

Maven

- A **build and project management** tool from the Apache Foundation, and the default in the Java world for twenty years
- Its bet is convention over configuration: Maven already knows where your sources live, where the tests are, and what a build does — you only describe what is *different*
- So a `pom.xml` is not a script. It is a **description of the project**, and Maven derives the build from it
- Which is why any Maven project, anywhere, is navigable by anyone who has seen one before

The price is rigidity: doing something Maven did not anticipate is genuinely awkward. That rigidity is also the reason ten thousand projects look alike.



current stable **3.9.16** (May 2026)
Maven 4 still in release candidates

Coordinates: the name of a thing

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.udc.fi.dc</groupId>        <!-- who made it  -->
  <artifactId>travel-planner</artifactId><!-- what it is   -->
  <version>1.0.0-SNAPSHOT</version>      <!-- which one    -->
  <packaging>jar</packaging>             <!-- what it becomes -->
  <name>Travel Planner</name>
</project>
```

- **`groupId`** names the organisation — by convention a domain read backwards. **`artifactId`** names the project. Together with **`version`** they are the coordinates, and they are **globally unique**: the same three strings mean the same bytes to everyone
- `version` is `major.minor.patch`. The suffix **`-SNAPSHOT`** means *still moving*: Maven re-checks it, and it is the only kind of version that may change under you
- `packaging` decides what is produced: `jar` (default), `war`, `pom`, `ear`

Coordinates: the name of a thing

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.udc.fi.dc</groupId>        <!-- who made it  -->
  <artifactId>travel-planner</artifactId><!-- what it is   -->
  <version>1.0.0-SNAPSHOT</version>      <!-- which one    -->
  <packaging>jar</packaging>             <!-- what it becomes -->
  <name>Travel Planner</name>
</project>
```

- **`groupId`** names the organisation — by convention a domain read backwards. **`artifactId`** names the project. Together with **`version`** they are the coordinates, and they are **globally unique**: the same three strings mean the same bytes to everyone
- `version` is `major.minor.patch`. The suffix **`-SNAPSHOT`** means *still moving*: Maven re-checks it, and it is the only kind of version that may change under you
- `packaging` decides what is produced: `jar` (default), `war`, `pom`, `ear`

A release version is a promise: once `1.0.0` is published, those bytes never change again. Everything else in this lesson rests on that promise.

The layout you do not get to choose



- Nothing here is configurable in practice — and that is the feature
- `src/` is **yours** and is versioned
- `target/` is **Maven's** and goes in `.gitignore`: it can be deleted at any moment and rebuilt
- Test code lives **beside** production code, in a mirror tree, so a class and its test share a package

The layout you do not get to choose



- Nothing here is configurable in practice — and that is the feature
- `src/` is **yours** and is versioned
- `target/` is **Maven's** and goes in `.gitignore`: it can be deleted at any moment and rebuilt
- Test code lives **beside** production code, in a mirror tree, so a class and its test share a package

About to commit something from `target/`? Stop: whatever you want to share should be produced by the build, not carried in the repository.

Parents and modules

The parent — inherit, do not repeat

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
<groupId>es.udc.fi.dc</groupId>
<artifactId>travel-planner</artifactId>
```

From the parent you inherit dependency *versions*, plugin configuration and properties — which is most of what a Spring Boot POM does not have to say.

```
<packaging>pom</packaging>
<modules>
  <module>travel-model</module>
  <module>travel-service</module>
  <module>travel-web</module>
</modules>
```

One `mvn verify` at the top builds all three, in dependency order. Each module produces exactly **one** artifact and has its own `pom.xml`.

Parents and modules

The parent — inherit, do not repeat

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
<groupId>es.udc.fi.dc</groupId>
<artifactId>travel-planner</artifactId>
```

From the parent you inherit dependency *versions*, plugin configuration and properties — which is most of what a Spring Boot POM does not have to say.

The aggregator — one build, many artifacts

```
<packaging>pom</packaging>
<modules>
  <module>travel-model</module>
  <module>travel-service</module>
  <module>travel-web</module>
</modules>
```

One `mvn verify` at the top builds all three, in dependency order. Each module produces exactly **one** artifact and has its own `pom.xml`.

Parents and modules

The parent — inherit, do not repeat

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
<groupId>es.udc.fi.dc</groupId>
<artifactId>travel-planner</artifactId>
```

From the parent you inherit dependency *versions*, plugin configuration and properties — which is most of what a Spring Boot POM does not have to say.

The aggregator — one build, many artifacts

```
<packaging>pom</packaging>
<modules>
  <module>travel-model</module>
  <module>travel-service</module>
  <module>travel-web</module>
</modules>
```

One `mvn verify` at the top builds all three, in dependency order. Each module produces exactly **one** artifact and has its own `pom.xml`.

The two are different jobs that often live in the same file: *parent* is about **inheriting configuration**, *modules* is about **building several things together**. A POM can be one, the other, or both.

Parents and modules

The parent — inherit, do not repeat

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
<groupId>es.udc.fi.dc</groupId>
<artifactId>travel-planner</artifactId>
```

From the parent you inherit dependency *versions*, plugin configuration and properties — which is most of what a Spring Boot POM does not have to say.

The aggregator — one build, many artifacts

```
<packaging>pom</packaging>
<modules>
  <module>travel-model</module>
  <module>travel-service</module>
  <module>travel-web</module>
</modules>
```

One `mvn verify` at the top builds all three, in dependency order. Each module produces exactly **one** artifact and has its own `pom.xml`.

The two are different jobs that often live in the same file: *parent* is about **inheriting configuration**, *modules* is about **building several things together**. A POM can be one, the other, or both.

Split into modules when a part of the system could be released on its own. Splitting because a folder felt big only buys you more `pom.xml` files to keep in step.

The lifecycle: you name the last step you want



The phases are **ordered**, and asking for one runs **everything before it**. `mvn package` validates, compiles, tests and then packages; there is no way to package without testing, and that is deliberate.

The lifecycle: you name the last step you want



The phases are **ordered**, and asking for one runs **everything before it**. `mvn package` validates, compiles, tests and then packages; there is no way to package without testing, and that is deliberate.

`clean` is not part of this sequence — it is a separate lifecycle that empties `target/`. Hence the habit: `mvn clean verify`.

The lifecycle: you name the last step you want



The phases are **ordered**, and asking for one runs **everything before it**. `mvn package` validates, compiles, tests and then packages; there is no way to package without testing, and that is deliberate.

`clean` is not part of this sequence — it is a separate lifecycle that empties `target/`. Hence the habit: `mvn clean verify`.

`verify` is the phase that matters for this course: it is the last one that does not touch anything outside your project, so it is what Lesson 5 will run on every push.

Maven does nothing; the plugins do everything

A phase is only a **name for a moment** in the build. The work is done by goals, and goals come from plugins — which are themselves artifacts, fetched by coordinates like anything else.



So *configuring the build* means, almost always, adding a plugin and binding one of its goals to a phase. `mvn surefire:test` runs the goal alone; `mvn test` runs the phase, and with it every goal bound to it.

Maven does nothing; the plugins do everything

A phase is only a **name for a moment** in the build. The work is done by goals, and goals come from plugins — which are themselves artifacts, fetched by coordinates like anything else.



So *configuring the build* means, almost always, adding a plugin and binding one of its goals to a phase. `mvn surefire:test` runs the goal alone; `mvn test` runs the phase, and with it every goal bound to it.

This is also why two Maven projects can behave very differently with almost identical POMs: what a build does is the sum of its plugins, and most of them arrive through the parent.

The commands you will actually type

COMMAND	WHEN
`mvn clean verify`	before every push. The one that matters
`mvn test`	while you work, dozens of times a day
`mvn -o verify`	offline: no network, only the local cache
`mvn -pl travel-web -am verify`	one module, plus what it needs
`mvn dependency:tree`	where on earth did *that* library come from
`mvn help:effective-pom`	the POM after inheritance — the real one
`mvn versions:display-dependency-updates`	what has moved since you pinned it
`mvn -DskipTests package`	when you know exactly why you are doing it

The commands you will actually type

COMMAND	WHEN
`mvn clean verify`	before every push. The one that matters
`mvn test`	while you work, dozens of times a day
`mvn -o verify`	offline: no network, only the local cache
`mvn -pl travel-web -am verify`	one module, plus what it needs
`mvn dependency:tree`	where on earth did *that* library come from
`mvn help:effective-pom`	the POM after inheritance — the real one
`mvn versions:display-dependency-updates`	what has moved since you pinned it
`mvn -DskipTests package`	when you know exactly why you are doing it

`help:effective-pom` is the one worth remembering. Your `pom.xml` is rarely the POM Maven uses: the parent, the plugins and the profiles all fold into it, and that command prints the result.

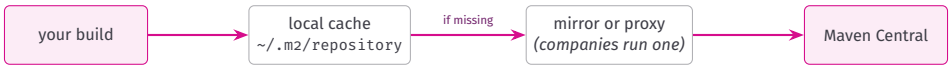
The commands you will actually type

COMMAND	WHEN
`mvn clean verify`	before every push. The one that matters
`mvn test`	while you work, dozens of times a day
`mvn -o verify`	offline: no network, only the local cache
`mvn -pl travel-web -am verify`	one module, plus what it needs
`mvn dependency:tree`	where on earth did *that* library come from
`mvn help:effective-pom`	the POM after inheritance — the real one
`mvn versions:display-dependency-updates`	what has moved since you pinned it
`mvn -DskipTests package`	when you know exactly why you are doing it

`help:effective-pom` is the one worth remembering. Your `pom.xml` is rarely the POM Maven uses: the parent, the plugins and the profiles all fold into it, and that command prints the result.

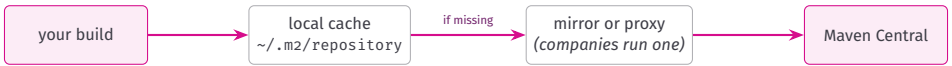
-DskipTests in a commit message is a confession. In a pipeline it is a defect.

Where the libraries come from



Maven looks in your **local cache** first. A release version found there is used without asking anyone — which is why a second build is fast, and why `mvn -o` works on a train.

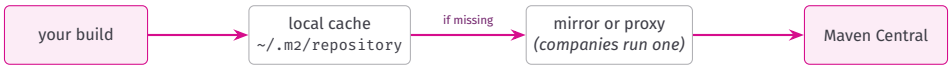
Where the libraries come from



Maven looks in your **local cache** first. A release version found there is used without asking anyone — which is why a second build is fast, and why `mvn -o` works on a train.

The cache is also where `mvn install` puts *your* artifacts, so another project on the same machine can depend on them. Convenient, and the single most common reason a build works for one person only.

Where the libraries come from



Maven looks in your **local cache** first. A release version found there is used without asking anyone — which is why a second build is fast, and why `mvn -o` works on a train.

The cache is also where `mvn install` puts *your* artifacts, so another project on the same machine can depend on them. Convenient, and the single most common reason a build works for one person only.

Central is immutable: "once a component has been released and published to the Central Repository, it cannot be altered." No republishing, no deleting — only a new version.

Declaring a dependency

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>                          <!-- version from the parent -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>                  <!-- not shipped -->
  </dependency>
</dependencies>
```

The scope answers *when is this on the classpath*:

SCOPE	MEANING
`compile`	the default: everywhere, and passed on to whoever depends on you
`provided`	you compile against it, but the container supplies it at run time
`runtime`	needed to run, not to compile — a JDBC driver, typically
`test`	tests only, and never transitive. JUnit, Mockito, Testcontainers

Declaring a dependency

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>                          <!-- version from the parent -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>                  <!-- not shipped -->
  </dependency>
</dependencies>
```

The scope answers *when is this on the classpath*:

SCOPE	MEANING
`compile`	the default: everywhere, and passed on to whoever depends on you
`provided`	you compile against it, but the container supplies it at run time
`runtime`	needed to run, not to compile — a JDBC driver, typically
`test`	tests only, and never transitive. JUnit, Mockito, Testcontainers

Getting scope right is not pedantry: a `compile`-scoped test library ends up inside your `.war`, on a production classpath, with its own dependencies in tow.

Properties and profiles

Properties — one place per fact

```
<properties>
  <java.version>21</java.version>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>
  <testcontainers.version>1.21.0</testcontainers.version>
</properties>
...
<version>${testcontainers.version}</version>
```

A version that appears twice will one day disagree with itself.

```
<profiles>
  <profile>
    <id>ci</id>
    <properties>
      <skip.frontend>false</skip.frontend>
    </properties>
  </profile>
</profiles>
```

Activated by hand (`mvn -Pci verify`), by a property, by an environment. **Not** a place to keep passwords.

Properties and profiles

Properties — one place per fact

```
<properties>
  <java.version>21</java.version>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>
  <testcontainers.version>1.21.0</testcontainers.version>
</properties>
...
<version>${testcontainers.version}</version>
```

A version that appears twice will one day disagree with itself.

Profiles — the same build, one thing changed

```
<profiles>
  <profile>
    <id>ci</id>
    <properties>
      <skip.frontend>false</skip.frontend>
    </properties>
  </profile>
</profiles>
```

Activated by hand (`mvn -Pci verify`), by a property, by an environment. **Not** a place to keep passwords.

Properties and profiles

Properties — one place per fact

```
<properties>
  <java.version>21</java.version>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>
  <testcontainers.version>1.21.0</testcontainers.version>
</properties>
...
<version>${testcontainers.version}</version>
```

A version that appears twice will one day disagree with itself.

Profiles — the same build, one thing changed

```
<profiles>
  <profile>
    <id>ci</id>
    <properties>
      <skip.frontend>false</skip.frontend>
    </properties>
  </profile>
</profiles>
```

Activated by hand (`mvn -Pci verify`), by a property, by an environment. **Not** a place to keep passwords.

Profiles are the sanctioned way to say *“on the server, do this instead”* — but every profile is a path that is not tested when you build locally. Two are useful; six mean nobody knows what the build does any more.

Properties and profiles

Properties — one place per fact

```
<properties>
  <java.version>21</java.version>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>
  <testcontainers.version>1.21.0</testcontainers.version>
</properties>
...
<version>${testcontainers.version}</version>
```

A version that appears twice will one day disagree with itself.

Profiles — the same build, one thing changed

```
<profiles>
  <profile>
    <id>ci</id>
    <properties>
      <skip.frontend>false</skip.frontend>
    </properties>
  </profile>
</profiles>
```

Activated by hand (`mvn -Pci verify`), by a property, by an environment. **Not** a place to keep passwords.

Profiles are the sanctioned way to say *"on the server, do this instead"* — but every profile is a path that is not tested when you build locally. Two are useful; six mean nobody knows what the build does any more.

Secrets never live in the POM, and the POM is in Git. Credentials go in `settings.xml` on the machine, or in the credentials store of the CI server — which is exactly what Lesson 5 will use.

Starting from an archetype

An archetype is a project template: a skeleton with the layout, the POM and some example code already in place. It is how an organisation says *“this is how we start a project here”*.

```
mvn archetype:generate \
    -DarchetypeGroupId=es.udc.fi.dc.fd -DarchetypeArtifactId=fd-spring-react \
    -DgroupId=es.udc.fi.dc -DartifactId=travel-planner
```

- Run without arguments it is interactive and offers you a few thousand archetypes, which is exactly as useful as it sounds. Name the one you want
- The archetype of this course gives you the **Spring Boot back end and the React front end** already wired together, building with one command
- `mvn archetype:create-from-project` turns a project you already have into a template — which is how yours would become next year’s starting point

Starting from an archetype

An archetype is a project template: a skeleton with the layout, the POM and some example code already in place. It is how an organisation says *“this is how we start a project here”*.

```
mvn archetype:generate \
    -DarchetypeGroupId=es.udc.fi.dc.fd -DarchetypeArtifactId=fd-spring-react \
    -DgroupId=es.udc.fi.dc -DartifactId=travel-planner
```

- Run without arguments it is interactive and offers you a few thousand archetypes, which is exactly as useful as it sounds. Name the one you want
- The archetype of this course gives you the **Spring Boot back end and the React front end** already wired together, building with one command
- `mvn archetype:create-from-project` turns a project you already have into a template — which is how yours would become next year’s starting point

An archetype is a starting point, not a framework: after the first minute the generated code is simply yours, and nothing updates it later.

SECTION 3

Dependencies

3

A dependency is a transfer of trust

Adding a line to `pom.xml` or `package.json` looks like configuration. It is not. It is a decision to **run somebody else's code**, on your machine and on your server, at every build from now on.

And not only theirs

You chose one library. It brought its own dependencies, which brought theirs. A modest Spring Boot service resolves **tens** of artifacts; a React front end routinely installs **hundreds to thousands** of packages. **You picked perhaps five of them.**

A dependency is a transfer of trust

Adding a line to `pom.xml` or `package.json` looks like configuration. It is not. It is a decision to **run somebody else's code**, on your machine and on your server, at every build from now on.

And not only theirs

You chose one library. It brought its own dependencies, which brought theirs. A modest Spring Boot service resolves **tens** of artifacts; a React front end routinely installs **hundreds to thousands** of packages. **You picked perhaps five of them.**

The uncomfortable question

How many of those authors do you know? How many have two-factor authentication on the account that can publish to the whole world under that name? You do not know — and their build is now part of yours.

A dependency is a transfer of trust

Adding a line to `pom.xml` or `package.json` looks like configuration. It is not. It is a decision to **run somebody else's code**, on your machine and on your server, at every build from now on.

And not only theirs

You chose one library. It brought its own dependencies, which brought theirs. A modest Spring Boot service resolves **tens** of artifacts; a React front end routinely installs **hundreds to thousands** of packages. **You picked perhaps five of them.**

The uncomfortable question

How many of those authors do you know? How many have two-factor authentication on the account that can publish to the whole world under that name? You do not know — and their build is now part of yours.

This is not an argument for writing everything yourself: that ends worse. It is an argument for knowing **what you depend on**, **which version**, and **how that version was chosen** — which is the rest of this section.

A dependency is a transfer of trust

Adding a line to `pom.xml` or `package.json` looks like configuration. It is not. It is a decision to **run somebody else's code**, on your machine and on your server, at every build from now on.

And not only theirs

You chose one library. It brought its own dependencies, which brought theirs. A modest Spring Boot service resolves **tens** of artifacts; a React front end routinely installs **hundreds to thousands** of packages. **You picked perhaps five of them.**

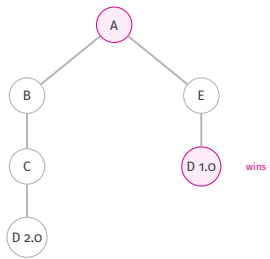
The uncomfortable question

How many of those authors do you know? How many have two-factor authentication on the account that can publish to the whole world under that name? You do not know — and their build is now part of yours.

This is not an argument for writing everything yourself: that ends worse. It is an argument for knowing **what you depend on**, **which version**, and **how that version was chosen** — which is the rest of this section.

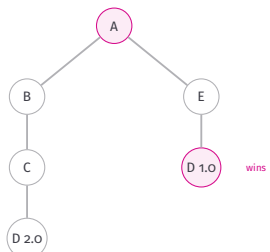
The industry name for this is the software supply chain, and lately it is where the interesting attacks happen.

How Maven picks a version



- **Nearest definition wins.** `D 1.0` is two steps from the root, `D 2.0` is three. Maven takes `D 1.0` — even though it is older
- **Ties go to whoever was declared first** in the POM
- **One version per artifact**, full stop. Maven builds a single classpath and there is exactly one `D` on it
- `dependencyManagement` **overrides** all of that: state the version there and it is the version, wherever it came from

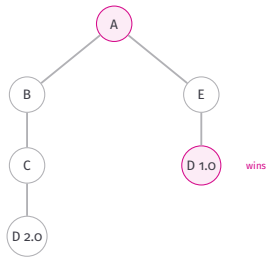
How Maven picks a version



- **Nearest definition wins.** `D 1.0` is two steps from the root, `D 2.0` is three. Maven takes `D 1.0` — even though it is older
- **Ties go to whoever was declared first** in the POM
- **One version per artifact**, full stop. Maven builds a single classpath and there is exactly one `D` on it
- `dependencyManagement` **overrides** all of that: state the version there and it is the version, wherever it came from

That last point is what a BOM is: a POM full of nothing but `dependencyManagement`, imported so that one authority decides the versions of a whole family. `spring-boot-starter-parent` is exactly this, and it is why your POM names so few versions.

How Maven picks a version



- **Nearest definition wins.** `D 1.0` is two steps from the root, `D 2.0` is three. Maven takes `D 1.0` — even though it is older
- **Ties go to whoever was declared first** in the POM
- **One version per artifact**, full stop. Maven builds a single classpath and there is exactly one `D` on it
- `dependencyManagement` **overrides** all of that: state the version there and it is the version, wherever it came from

That last point is what a BOM is: a POM full of nothing but `dependencyManagement`, imported so that one authority decides the versions of a whole family. `spring-boot-starter-parent` is exactly this, and it is why your POM names so few versions.

When something behaves like the wrong version, it usually is: `mvn dependency:tree` prints the whole graph and marks what was omitted, and why.

How npm picks a version — differently

```
"dependencies": {
  "react":  "^19.2.0",   // >= 19.2.0  and  < 20.0.0   -- minor and patch may move
  "lodash": "~4.17.21",  // >= 4.17.21 and  < 4.18.0   -- patch may move
  "axios":  "1.8.4"      // exactly this one
}
```

- npm dependencies are **ranges**, not versions. ^ is the default, and it is what `npm install <pkg>` writes for you
- So "*the same `package.json`*" does **not** mean the same code: install it today and in three weeks and you may get different bytes
- And where Maven forces one version, npm **nests**: two libraries wanting incompatible versions of `lodash` each get their own copy inside `node_modules`. Nothing breaks — the tree just grows

How npm picks a version — differently

```
"dependencies": {
  "react":  "^19.2.0",   // >= 19.2.0  and  < 20.0.0   -- minor and patch may move
  "lodash": "~4.17.21",  // >= 4.17.21 and  < 4.18.0   -- patch may move
  "axios":  "1.8.4"      // exactly this one
}
```

- npm dependencies are **ranges**, not versions. ^ is the default, and it is what `npm install <pkg>` writes for you
- So "*the same `package.json`*" does **not** mean the same code: install it today and in three weeks and you may get different bytes
- And where Maven forces one version, npm **nests**: two libraries wanting incompatible versions of `lodash` each get their own copy inside `node_modules`. Nothing breaks — the tree just grows

Both designs are defensible. Maven's gives you one predictable classpath and occasional impossible conflicts; npm's never conflicts and gives you a directory with thousands of packages in it, of which you have read none.

How npm picks a version — differently

```
"dependencies": {
  "react":  "^19.2.0",   // >= 19.2.0  and  < 20.0.0   -- minor and patch may move
  "lodash": "~4.17.21",  // >= 4.17.21 and  < 4.18.0   -- patch may move
  "axios":  "1.8.4"      // exactly this one
}
```

- npm dependencies are **ranges**, not versions. ^ is the default, and it is what `npm install <pkg>` writes for you
- So *"the same `package.json`"* does **not** mean the same code: install it today and in three weeks and you may get different bytes
- And where Maven forces one version, npm **nests**: two libraries wanting incompatible versions of `lodash` each get their own copy inside `node_modules`. Nothing breaks — the tree just grows

Both designs are defensible. Maven's gives you one predictable classpath and occasional impossible conflicts; npm's never conflicts and gives you a directory with thousands of packages in it, of which you have read none.

The consequence matters more than the mechanism: with ranges, a build that worked yesterday can fail today without anyone touching the repository.

The lockfile is the build's memory

`package.json` says what you **want**; `package-lock.json` records what you **got** — the exact version of every package in the whole tree, with a checksum of its contents. It is generated, it is ugly, and it belongs **in the repository**.

```
npm install     # resolves the ranges, MAY update the lockfile, writes node_modules
npm ci          # installs exactly the lockfile. Fails if it disagrees with package.json.
                # Deletes node_modules first. Never writes to the lockfile.
```

- `npm ci` is the command for **CI and for every fresh clone**. `npm install` is for the moment you deliberately change a dependency
- Maven has no lockfile because it does not need one: versions are exact, and Central is immutable, so the same POM resolves to the same bytes forever
- Except for `-SNAPSHOT` versions, which are the one place Maven behaves like npm — and the reason nothing you release should depend on one

The lockfile is the build's memory

`package.json` says what you **want**; `package-lock.json` records what you **got** — the exact version of every package in the whole tree, with a checksum of its contents. It is generated, it is ugly, and it belongs **in the repository**.

```
npm install     # resolves the ranges, MAY update the lockfile, writes node_modules
npm ci          # installs exactly the lockfile. Fails if it disagrees with package.json.
                # Deletes node_modules first. Never writes to the lockfile.
```

- `npm ci` is the command for **CI and for every fresh clone**. `npm install` is for the moment you deliberately change a dependency
- Maven has no lockfile because it does not need one: versions are exact, and Central is immutable, so the same POM resolves to the same bytes forever
- Except for `-SNAPSHOT` versions, which are the one place Maven behaves like npm — and the reason nothing you release should depend on one

A lockfile left out of the repository, or a pipeline that runs `npm install`, throws away the only record of what your application is made of. (TripWise uses pnpm: `pnpm-lock.yaml` and `pnpm install --frozen-lockfile`, the same idea.)

What actually goes wrong

WHEN	WHAT	HOW IT REACHED YOU
2016	`left-pad`, eleven lines	The author unpublished it. Thousands of builds worldwide stopped compiling that afternoon
Sep 2025	`chalk`, `debug` and 16 more, together over **2 billion downloads a week**	A maintainer was phished by a fake `npmjs.help` two-factor reset. New patch versions carried a browser wallet-drainer for about two hours
Sep–Nov 2025	**Shai-Hulud**: over 500 packages, then 25,000 malicious repositories	A **self-replicating** install script: steal the maintainer's tokens, publish itself into their other packages, repeat
Aug 2026	`keyv`, `flat-cache`, `cacheable` and others — 1,300+ versions	The same worm, a new variant, running **before** install this time

What actually goes wrong

WHEN	WHAT	HOW IT REACHED YOU
2016	`left-pad`, eleven lines	The author unpublished it. Thousands of builds worldwide stopped compiling that afternoon
Sep 2025	`chalk`, `debug` and 16 more, together over **2 billion downloads a week**	A maintainer was phished by a fake `npmjs.help` two-factor reset. New patch versions carried a browser wallet-drainer for about two hours
Sep–Nov 2025	**Shai-Hulud**: over 500 packages, then 25,000 malicious repositories	A **self-replicating** install script: steal the maintainer's tokens, publish itself into their other packages, repeat
Aug 2026	`keyv`, `flat-cache`, `cacheable` and others — 1,300+ versions	The same worm, a new variant, running **before** install this time

The pattern does not change: **phish a maintainer, publish a patch release, wait**. Your caret range accepts it, your next install downloads it, and an npm package may **run a script at install time** — with your user's permissions, on your laptop, or on the CI server where the deployment keys live.

What you can actually do about it

In the project

- Commit the **lockfile**; build with `npm ci` everywhere except when you are deliberately upgrading
- **Pin** what you can, and upgrade **on purpose** — in a merge request, with the pipeline green, not silently at 2 a.m.
- Run `npm audit` and `mvn dependency:tree`; let **Dependabot** or **Renovate** file the upgrade
- Fewer dependencies. A package that saves four lines is not free

In your habits

- Two-factor authentication on **GitLab, GitHub and npm**. Every one of these attacks began with a stolen credential
- A token is a password. Not in the repository, not in the POM, not in a screenshot
- If a machine ran a compromised install, assume its tokens are gone and **rotate them**
- `npm ci –ignore-scripts` where the project allows it

What you can actually do about it

In the project

- Commit the **lockfile**; build with `npm ci` everywhere except when you are deliberately upgrading
- **Pin** what you can, and upgrade **on purpose** — in a merge request, with the pipeline green, not silently at 2 a.m.
- Run `npm audit` and `mvn dependency:tree`; let **Dependabot** or **Renovate** file the upgrade
- Fewer dependencies. A package that saves four lines is not free

In your habits

- Two-factor authentication on **GitLab, GitHub and npm**. Every one of these attacks began with a stolen credential
- A token is a password. Not in the repository, not in the POM, not in a screenshot
- If a machine ran a compromised install, assume its tokens are gone and **rotate them**
- `npm ci –ignore-scripts` where the project allows it

The registries moved too: after 2025 npm **revoked every classic token**, made two-factor the default for publishing, and now recommends **trusted publishing** — the package is published by a pipeline proving its identity, with no long-lived token to steal.

What you can actually do about it

In the project

- Commit the **lockfile**; build with `npm ci` everywhere except when you are deliberately upgrading
- **Pin** what you can, and upgrade **on purpose** — in a merge request, with the pipeline green, not silently at 2 a.m.
- Run `npm audit` and `mvn dependency:tree`; let **Dependabot** or **Renovate** file the upgrade
- Fewer dependencies. A package that saves four lines is not free

In your habits

- Two-factor authentication on **GitLab, GitHub and npm**. Every one of these attacks began with a stolen credential
- A token is a password. Not in the repository, not in the POM, not in a screenshot
- If a machine ran a compromised install, assume its tokens are gone and **rotate them**
- `npm ci –ignore-scripts` where the project allows it

The registries moved too: after 2025 npm **revoked every classic token**, made two-factor the default for publishing, and now recommends **trusted publishing** — the package is published by a pipeline proving its identity, with no long-lived token to steal.

None of this is paranoia. It is the same reasoning as the rest of the subject: make the machine remember, so a human being having a bad day cannot lose it.

SECTION 4

Gradle



Gradle

- The other build tool you will meet: general purpose, and the standard for **Android** and much of the Kotlin world
- Where a POM *describes* a project, a Gradle build file is **a program** — Kotlin or Groovy — that configures a build
- So there is no fixed lifecycle. A build is a **graph of tasks** with dependencies between them, and plugins add the tasks
- Which buys speed: Gradle knows each task's inputs and outputs, so it **skips what has not changed** and caches the rest



current stable **9.7.1** (August 2026)
Kotlin is now the default DSL

Gradle

- The other build tool you will meet: general purpose, and the standard for **Android** and much of the Kotlin world
- Where a POM *describes* a project, a Gradle build file is **a program** — Kotlin or Groovy — that configures a build
- So there is no fixed lifecycle. A build is a **graph of tasks** with dependencies between them, and plugins add the tasks
- Which buys speed: Gradle knows each task's inputs and outputs, so it **skips what has not changed** and caches the rest

The trade is the mirror image of Maven's: more power, less predictability. A Gradle build can do anything, which includes becoming something only its author understands.



current stable **9.7.1** (August 2026)
Kotlin is now the default DSL

The build file, in today's Gradle

```
plugins {
    java
    id("org.springframework.boot") version "4.1.0"
}
repositories { mavenCentral() }
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    testImplementation("org.junit.jupiter:junit-jupiter")
}
```

- The file is `build.gradle.kts` — **Kotlin** is the default for new builds, and the editor can complete it, which Groovy could not
- Coordinates are the same `group:artifact:version` as Maven, because it is the same Maven Central underneath
- The scope is called a configuration: `implementation` (yours alone), `api` (visible to whoever depends on you), `compileOnly`, `runtimeOnly`, `testImplementation`

The build file, in today's Gradle

```
plugins {
    java
    id("org.springframework.boot") version "4.1.0"
}
repositories { mavenCentral() }
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    testImplementation("org.junit.jupiter:junit-jupiter")
}
```

- The file is `build.gradle.kts` — **Kotlin** is the default for new builds, and the editor can complete it, which Groovy could not
- Coordinates are the same `group:artifact:version` as Maven, because it is the same Maven Central underneath
- The scope is called a configuration: `implementation` (yours alone), `api` (visible to whoever depends on you), `compileOnly`, `runtimeOnly`, `testImplementation`

`compile` and `testCompile` in a tutorial date it to before Gradle 7: those keywords no longer exist. `implementation` is the one you want.

Two things every Gradle project should have

A version catalog

`gradle/libs.versions.toml`

```
[versions]
springBoot = "4.1.0"

[libraries]
spring-web = { module = "org.springframework.boot:spring-boot-
      starter-web",
              version.ref = "springBoot" }
```

Then `implementation(libs.spring.web)` everywhere. One file holds every version in the build — Gradle's answer to `dependencyManagement`.

`gradlew`, `gradlew.bat`, `gradle/wrapper/`

```
./gradlew build      # not: gradle build
```

A script, committed to the repository, that downloads the **exact Gradle version this project was built with** and runs it. Nobody has to install Gradle; nobody can use the wrong one.

Maven has the same idea — `mvnw` — and it is worth using for exactly the same reason.

Two things every Gradle project should have

A version catalog

`gradle/libs.versions.toml`

```
[versions]
springBoot = "4.1.0"

[libraries]
spring-web = { module = "org.springframework.boot:spring-boot-
      starter-web",
              version.ref = "springBoot" }
```

Then `implementation(libs.spring.web)` everywhere. One file holds every version in the build — Gradle's answer to `dependencyManagement`.

The wrapper

`gradlew`, `gradlew.bat`, `gradle/wrapper/`

```
./gradlew build      # not: gradle build
```

A script, committed to the repository, that downloads the **exact Gradle version this project was built with** and runs it. Nobody has to install Gradle; nobody can use the wrong one.

Maven has the same idea — `mvnw` — and it is worth using for exactly the same reason.

Two things every Gradle project should have

A version catalog

`gradle/libs.versions.toml`

```
[versions]
springBoot = "4.1.0"

[libraries]
spring-web = { module = "org.springframework.boot:spring-boot-
      starter-web",
              version.ref = "springBoot" }
```

Then `implementation(libs.spring.web)` everywhere. One file holds every version in the build — Gradle's answer to `dependencyManagement`.

The wrapper

`gradlew`, `gradlew.bat`, `gradle/wrapper/`

```
./gradlew build      # not: gradle build
```

A script, committed to the repository, that downloads the **exact Gradle version this project was built with** and runs it. Nobody has to install Gradle; nobody can use the wrong one.

Maven has the same idea — `mvnw` — and it is worth using for exactly the same reason.

Both are the same instinct as the lockfile: take a fact that lives in somebody's head or in somebody's installation, and write it down in the repository where the build can read it.

The same ideas, in the other vocabulary

IDEA	MAVEN	GRADLE
the build file	`pom.xml`	`build.gradle.kts`
what a build is	a fixed lifecycle of phases	a graph of tasks
run everything	`mvn clean verify`	`./gradlew clean build`
list what you can run	`mvn help:describe`	`./gradlew tasks`
declare a dependency	`<dependency>`, with a scope	`implementation("...")`
centralise versions	`dependencyManagement`, BOM	version catalog, `platform()`
pin the tool itself	`mvnw`	`gradlew` *(expected, not optional)*
start a project	`archetype:generate`	`gradle init`
the local cache	`~/.m2/repository`	`~/.gradle/caches`

The same ideas, in the other vocabulary

IDEA	MAVEN	GRADLE
the build file	`pom.xml`	`build.gradle.kts`
what a build is	a fixed lifecycle of phases	a graph of tasks
run everything	`mvn clean verify`	`./gradlew clean build`
list what you can run	`mvn help:describe`	`./gradlew tasks`
declare a dependency	`<dependency>`, with a scope	`implementation("...")`
centralise versions	`dependencyManagement`, BOM	version catalog, `platform()`
pin the tool itself	`mvnw`	`gradlew` *(expected, not optional)*
start a project	`archetype:generate`	`gradle init`
the local cache	`~/.m2/repository`	`~/.gradle/caches`

Which is the point of putting them side by side. You are not learning Maven; you are learning what a build tool is, in a syntax that happens to be XML this year.

The same ideas, in the other vocabulary

IDEA	MAVEN	GRADLE
the build file	`pom.xml`	`build.gradle.kts`
what a build is	a fixed lifecycle of phases	a graph of tasks
run everything	`mvn clean verify`	`./gradlew clean build`
list what you can run	`mvn help:describe`	`./gradlew tasks`
declare a dependency	`<dependency>`, with a scope	`implementation("...")`
centralise versions	`dependencyManagement`, BOM	version catalog, `platform()`
pin the tool itself	`mvnw`	`gradlew` *(expected, not optional)*
start a project	`archetype:generate`	`gradle init`
the local cache	`~/.m2/repository`	`~/.gradle/caches`

Which is the point of putting them side by side. You are not learning Maven; you are learning what a build tool is, in a syntax that happens to be XML this year.

Choose Maven when you want a build anyone can read, and Gradle when the build is genuinely complicated or the compile times hurt. For a project the size of yours, Maven and an early night.

SECTION 5

Working this way

5

Your build, seen from the pipeline

Next lesson this becomes literal. A pipeline is not a clever piece of software: it is a machine that clones your branch and runs your build, and then believes the **exit code**.

```
git clone ... && git checkout feat/42-trip-sharing
mvn -B clean verify          # -B: batch mode, no colours, no questions
echo $?                      # 0 -> green.  anything else -> red, and the MR is blocked
```

- So everything the pipeline reports has to be produced **by the build**: the test results in `target/surefire-reports/`, the coverage, the `.jar` in `target/`
- A build that asks a question, needs a file that is not in the repository, or depends on something you installed by hand **hangs or fails** — and nobody is there to help it
- And a build that passes locally but fails there is not the server being difficult. It is your build depending on your laptop

Your build, seen from the pipeline

Next lesson this becomes literal. A pipeline is not a clever piece of software: it is a machine that clones your branch and runs your build, and then believes the **exit code**.

```
git clone ... && git checkout feat/42-trip-sharing
mvn -B clean verify          # -B: batch mode, no colours, no questions
echo $?                      # 0 -> green.  anything else -> red, and the MR is blocked
```

- So everything the pipeline reports has to be produced **by the build**: the test results in `target/surefire-reports/`, the coverage, the `.jar` in `target/`
- A build that asks a question, needs a file that is not in the repository, or depends on something you installed by hand **hangs or fails** — and nobody is there to help it
- And a build that passes locally but fails there is not the server being difficult. It is your build depending on your laptop

The whole of Lesson 5 rests on one sentence from the start of this one: if it is not in the build, it does not exist.

The rules, for the assignment

The build

- **One command builds everything**: `mvn clean verify` at the root, back end and front end, on a clean clone
- **No manual step.** Nothing installed by hand into anyone's `.m2`, nothing that only works from the IDE
- The build **fails when a test fails**. No `-DskipTests`, no disabled tests to make the pipeline green
- **Same Java, same Maven** for the whole team — written down, not agreed verbally

The dependencies

- **No floating versions.** Every version comes from the POM, a property or the parent — never from chance
- `package-lock.json` is **committed**, and CI uses `npm ci`
- **No binaries in Git**: no `.jar`, no `target/`, no `node_modules/`
- Adding a dependency is a **decision**: it goes in a merge request and somebody reviews it, like any other change

The rules, for the assignment

The build

- **One command builds everything**: `mvn clean verify` at the root, back end and front end, on a clean clone
- **No manual step.** Nothing installed by hand into anyone's `.m2`, nothing that only works from the IDE
- The build **fails when a test fails**. No `-DskipTests`, no disabled tests to make the pipeline green
- **Same Java, same Maven** for the whole team — written down, not agreed verbally

The dependencies

- **No floating versions.** Every version comes from the POM, a property or the parent — never from chance
- `package-lock.json` is **committed**, and CI uses `npm ci`
- **No binaries in Git**: no `.jar`, no `target/`, no `node_modules/`
- Adding a dependency is a **decision**: it goes in a merge request and somebody reviews it, like any other change

All of it is checked the same way: we clone your repository onto a machine that has never seen your project, and run one command. That is also, exactly, what happens to real software.

Your turn

1. Clone your team's repository into an **empty folder** and run `mvn clean verify`. If it does not work, you have found your first issue of the iteration
2. Run `mvn dependency:tree` and find a library **nobody on the team chose**. Work out who brought it in
3. Run `mvn help:effective-pom` and look for something you did not know was in your build
4. Count the packages in your front end's `node_modules`, then count the dependencies in `package.json`. Discuss
5. Make sure `package-lock.json` is committed, and change the pipeline step to `npm ci`
6. Pin the Java version in the POM, and write in the team wiki which JDK and which Maven everybody is using

Your turn

1. Clone your team's repository into an **empty folder** and run `mvn clean verify`. If it does not work, you have found your first issue of the iteration
2. Run `mvn dependency:tree` and find a library **nobody on the team chose**. Work out who brought it in
3. Run `mvn help:effective-pom` and look for something you did not know was in your build
4. Count the packages in your front end's `node_modules`, then count the dependencies in `package.json`. Discuss
5. Make sure `package-lock.json` is committed, and change the pipeline step to `npm ci`
6. Pin the Java version in the POM, and write in the team wiki which JDK and which Maven everybody is using

The second and fourth are the ones to do together, out loud. They are usually the moment the previous section stops being an abstraction.

What to take away

The build is a program. It lives in the repository, it is reviewed like code, and it is the only definition of what your project is.

Convention beats configuration. Maven's rigidity is why any Java developer can find their way around your project in a minute.

A phase is a moment; plugins do the work. Everything Maven appears to do natively is an artifact fetched by coordinates.

A dependency is code you agreed to run. Know what you depend on, which version, and why that version — and commit the lockfile.

If it is not in the build, it does not exist. Not in your head, not in your IDE, not in a command somebody remembers.

What to take away

The build is a program. It lives in the repository, it is reviewed like code, and it is the only definition of what your project is.

Convention beats configuration. Maven's rigidity is why any Java developer can find their way around your project in a minute.

A phase is a moment; plugins do the work. Everything Maven appears to do natively is an artifact fetched by coordinates.

A dependency is code you agreed to run. Know what you depend on, which version, and why that version — and commit the lockfile.

If it is not in the build, it does not exist. Not in your head, not in your IDE, not in a command somebody remembers.

Next: *Lesson 5* — a server that runs all of this for you, on every push, and tells everyone when it breaks.

Appendix: the three lifecycles, in full

clean

pre-clean
clean
post-clean

site

pre-site
site
post-site
site-deploy

default — the phases in bold are the ones you name

validate
initialize
generate-sources
process-sources
generate-resources
process-resources
compile
process-classes
generate-test-sources
process-test-sources
generate-test-resources
process-test-resources
test-compile
process-test-classes
test
prepare-package
package
pre-integration-test
integration-test
post-integration-test
verify
install
deploy

Appendix: the plugins you will meet

PLUGIN	BOUND TO	WHAT IT DOES
`compiler`	`compile`	compiles Java; where the language level lives
`surefire`	`test`	runs the unit tests, writes `target/surefire-reports`
`failsafe`	`integration-test`	runs `*IT` tests; fails the build at `verify`
`jar / war`	`package`	builds the artifact
`spring-boot`	`package`	repackages it into an executable jar
`install / deploy`	same-named phases	publishes it, locally or remotely
`clean`	`clean`	empties `target/`
`jacoco`	`test`, `verify`	measures coverage — *Lesson 6*
`sonar`	invoked directly	sends the analysis — *Lesson 6*
`frontend`	`generate-resources`	runs npm from Maven, for the React build
`versions`	invoked directly	reports and updates dependency versions
`dependency`	invoked directly	`tree`, `analyze`, `go-offline`

Appendix: the rest of the POM

```
<build>
  <finalName>travel-planner</finalName>      <!-- name of the artifact -->
  <resources>...</resources>                 <!-- what gets copied, and filtered -->
  <plugins>
    <plugin>
      <groupId>...</groupId><artifactId>...</artifactId><version>...</version>
      <configuration>...</configuration>     <!-- specific to the plugin -->
      <executions><execution>
         <phase>verify</phase><goals><goal>check</goal></goals>
      </execution></executions>              <!-- bind a goal to a phase -->
    </plugin>
  </plugins>
</build>
```

Sections worth knowing exist: `<scm>` (where the code lives), `<issueManagement>` (*Lesson 3*), `<ciManagement>` (*Lesson 5*), `<distributionManagement>` (where `deploy` publishes), `<reporting>` (the generated project site), `<licenses>`, `<developers>`.

The complete reference is `maven.apache.org/pom.html`. Nobody writes a POM from it; you read it when something surprises you.

Appendix: the commands, side by side

WHAT YOU WANT	MAVEN	NPM
install dependencies	happens during the build	`npm ci`
add one	edit the POM	`npm install <pkg>`
compile	`mvn compile`	`npm run build`
test	`mvn test`	`npm test`
everything, for CI	`mvn -B clean verify`	`npm ci && npm run build`
see the tree	`mvn dependency:tree`	`npm ls –all`
known vulnerabilities	`mvn dependency-check:check`	`npm audit`
what is out of date	`mvn versions:display-dependency-updates`	`npm outdated`
work offline	`mvn -o verify`	`npm ci –offline`
start from nothing	`mvn archetype:generate`	`npm create vite@latest`

Both halves of your project, in one table. The row that matters is the fifth: that is the line the pipeline runs.